



# UNIVERSIDADE LICUNGO

Faculdade de Ciências e Tecnologias  
Curso de Licenciatura em Engenharia Informática

## LABORATÓRIO 1

# Sistema de Estatística de Listas em Haskell

utilizando Linux, Git e GitHub

**Grupo:** Grupo 4

**Docente:** Daniel Gimo

**Estudantes:** Acácio Rombe Machica  
Alberto Lucas Alface  
Cláudio Armando Chaúque  
Pedro Francisco Antonio

Beira, 10 de Julho de 2026



## UNIVERSIDADE LICUNGO

Faculdade de Ciências e Tecnologias  
Curso de Licenciatura em Engenharia Informática

# Sistema de Estatística de Listas em Haskell

utilizando Linux, Git e GitHub

Trabalho prático apresentado no âmbito da Cadeira de Laboratório 1, do Curso de Licenciatura em Engenharia Informática da Universidade Licungo, como requisito parcial de avaliação, sob orientação do Docente Daniel Gimo.

**Grupo:** Grupo 4

**Docente:** Daniel Gimo

**Estudantes:** Acácio Rombe Machica  
Alberto Lucas Alface  
Cláudio Armando Chaúque  
Pedro Francisco Antonio

Beira, 10 de Julho de 2026

# Declaração de Autoria

Declaramos, para os devidos efeitos, que o presente trabalho académico intitulado "*Sistema de Estatística de Listas em Haskell utilizando Linux, Git e GitHub*" é de nossa autoria exclusiva e foi realizado no âmbito da disciplina de [INSERIR DISCIPLINA], do curso de Licenciatura em Engenharia Mecatrónica da [NOME DA UNIVERSIDADE].

Afirmamos que todo o código-fonte, documentação, análise e conteúdo textual deste relatório foram elaborados pelo Grupo 4, sem recurso a plágios ou reprodução não autorizada de obras alheias, exceto nas citações devidamente referenciadas segundo as normas académicas vigentes.

Comprometemo-nos a assumir plena responsabilidade académica pelo conteúdo aqui apresentado.

Beira, 2026

---

Assinatura dos membros do Grupo 4

# Agradecimentos

Os membros do Grupo 4 expressam a sua profunda gratidão a todas as pessoas e instituições que tornaram possível a realização deste trabalho.

Em primeiro lugar, agradecemos ao(à) Prof.(a) [INSERIR NOME], cujas orientações e ensinamentos foram fundamentais para a compreensão dos conceitos de programação funcional, sistemas operativos Linux e controlo de versões com Git e GitHub.

Agradecemos igualmente à [NOME DA UNIVERSIDADE] e à Faculdade de Ciências e Tecnologias pela infraestrutura académica disponibilizada, bem como pelo ambiente propício ao desenvolvimento do conhecimento científico e tecnológico.

Expressamos também o nosso reconhecimento às comunidades abertas do Haskell, Linux e Git, cujas documentações, tutoriais e fóruns foram fontes de consulta indispensáveis ao longo do desenvolvimento deste projeto.

Por fim, agradecemos às nossas famílias e colegas pelo apoio incondicional prestado durante todo o percurso académico.

*Grupo 4 — Beira, 2026*

# Resumo

O presente relatório documenta o desenvolvimento de um Sistema de Estatística de Listas, implementado na linguagem de programação funcional Haskell, em ambiente Linux, com controlo de versões gerido pelo sistema Git e hospedagem remota no GitHub. O projeto foi desenvolvido no âmbito da disciplina de [INSERIR DISCIPLINA], do curso de Licenciatura em Engenharia Mecatrónica da [NOME DA UNIVERSIDADE], pelo Grupo 4, como trabalho prático integrador das competências adquiridas ao longo do semestre.

O principal objetivo do sistema desenvolvido consiste na leitura de uma lista de números inteiros ou reais introduzidos pelo utilizador através do terminal Linux, e no processamento estatístico dessa lista, produzindo como resultado cinco métricas fundamentais: a soma total dos elementos, a média aritmética, o maior valor presente na lista, o menor valor presente na lista e a quantidade total de elementos. A aplicação é executada em modo de linha de comando, interagindo com o utilizador de forma simples e intuitiva.

No que respeita à metodologia de desenvolvimento, adoptou-se um modelo de desenvolvimento incremental com recurso a ramificações (*branches*) no sistema de controlo de versões Git. O projeto foi estruturado em quatro fases distintas, cada uma correspondente a uma *branch* dedicada: a *branch master* (estrutura base e captura de dados), a *branch fase1* (funções de soma e média), a *branch fase2* (funções de maior e menor valor) e a *branch fim* (função de contagem de elementos e interface final). Cada fase foi integrada na *branch* principal através de operações de *merge* do tipo *fast-forward*, resultando num histórico de commits limpo e documentado.

Os resultados alcançados demonstram a viabilidade e a adequação da linguagem Haskell para o desenvolvimento de aplicações de processamento de dados, evidenciando as vantagens do paradigma funcional em termos de concisão, clareza e segurança de tipos. Os testes realizados com entradas variadas, incluindo listas com um único elemento, listas com dois elementos e listas extensas com 28 números, confirmaram a correção dos algoritmos implementados. Foi ainda testado o comportamento do sistema perante entradas inválidas, verificando-se a geração da exceção `Prelude.read: no parse` pela função de leitura do Haskell.

O projeto contribuiu significativamente para o desenvolvimento das competências do Grupo 4 em programação funcional com Haskell, gestão de projetos com Git e

GitHub e utilização profissional do terminal Linux.

**Palavras-chave:** Haskell, Linux, Git, GitHub, Estatística, Programação Funcional.

# Abstract

This report documents the development of a List Statistics System, implemented in the functional programming language Haskell, in a Linux environment, with version control managed by the Git system and remote hosting on GitHub. The project was developed within the scope of the course [INSERT COURSE NAME], of the Bachelor's degree in Mechatronic Engineering at [UNIVERSITY NAME], by Group 4, as an integrative practical assignment of the competencies acquired throughout the semester.

The main objective of the developed system consists of reading a list of integer or real numbers entered by the user through the Linux terminal, and statistically processing that list, producing as output five fundamental metrics: the total sum of the elements, the arithmetic mean, the largest value in the list, the smallest value in the list, and the total number of elements. The application runs in command-line mode, interacting with the user in a simple and intuitive manner.

Regarding the development methodology, an incremental development model was adopted using branches in the Git version control system. The project was structured in four distinct phases, each corresponding to a dedicated branch: the `master` branch (base structure and data capture), the `fase1` branch (sum and mean functions), the `fase2` branch (maximum and minimum value functions), and the `fin` branch (element counting function and final interface). Each phase was integrated into the main branch through fast-forward merge operations, resulting in a clean and documented commit history.

The results achieved demonstrate the feasibility and suitability of the Haskell language for developing data processing applications, highlighting the advantages of the functional paradigm in terms of conciseness, clarity, and type safety. Tests performed with varied inputs, including lists with a single element, lists with two elements, and extensive lists with 28 numbers, confirmed the correctness of the implemented algorithms. The system's behavior in the presence of invalid input was also tested, verifying the generation of the `Prelude.read: no parse` exception by Haskell's `read` function.

**Keywords:** Haskell, Linux, Git, GitHub, Statistics, Functional Programming.

# Conteúdo

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>Declaração de Autoria</b>                                        | <b>2</b>  |
| <b>Agradecimentos</b>                                               | <b>3</b>  |
| <b>Resumo</b>                                                       | <b>4</b>  |
| <b>Abstract</b>                                                     | <b>6</b>  |
| <b>1 Introdução</b>                                                 | <b>13</b> |
| 1.1 Contextualização . . . . .                                      | 13        |
| 1.1.1 Importância da Programação Funcional . . . . .                | 13        |
| 1.1.2 Importância do Haskell . . . . .                              | 13        |
| 1.1.3 Importância do Git . . . . .                                  | 14        |
| 1.1.4 Importância do Linux no Desenvolvimento de Software . . . . . | 14        |
| 1.2 Problema . . . . .                                              | 14        |
| 1.3 Objetivo Geral . . . . .                                        | 15        |
| 1.4 Objetivos Específicos . . . . .                                 | 15        |
| 1.5 Metodologia . . . . .                                           | 15        |
| <b>2 Fundamentação Teórica</b>                                      | <b>17</b> |
| 2.1 Haskell . . . . .                                               | 17        |
| 2.1.1 História . . . . .                                            | 17        |
| 2.1.2 Características Fundamentais . . . . .                        | 17        |
| 2.1.3 Paradigma Funcional . . . . .                                 | 18        |
| 2.1.4 Tipagem Forte . . . . .                                       | 18        |
| 2.2 Linux . . . . .                                                 | 18        |
| 2.2.1 Sistema Operativo . . . . .                                   | 18        |
| 2.2.2 Terminal e Shell . . . . .                                    | 19        |
| 2.2.3 Gestão de Ficheiros . . . . .                                 | 19        |
| 2.3 Git . . . . .                                                   | 19        |
| 2.4 GitHub . . . . .                                                | 20        |
| <b>3 Desenvolvimento do Projeto</b>                                 | <b>21</b> |
| 3.1 Criação do Ambiente de Trabalho . . . . .                       | 21        |
| 3.2 Implementação da Estrutura Inicial . . . . .                    | 23        |

|          |                                                                                       |           |
|----------|---------------------------------------------------------------------------------------|-----------|
| 3.3      | Desenvolvimento da Branch <code>fase1</code> . . . . .                                | 24        |
| 3.3.1    | Criação da Branch e Conceito de Ramificação . . . . .                                 | 24        |
| 3.3.2    | Implementação de <code>calcularSoma</code> e <code>calcularMedia</code> . . . . .     | 25        |
| 3.4      | Integração da Branch <code>fase1</code> na Branch Principal . . . . .                 | 26        |
| 3.5      | Desenvolvimento da Branch <code>fase2</code> . . . . .                                | 26        |
| 3.5.1    | Implementação das Funções <code>obterMaior</code> e <code>obterMenor</code> . . . . . | 27        |
| 3.6      | Integração da Branch <code>fase2</code> . . . . .                                     | 27        |
| 3.7      | Desenvolvimento da Branch <code>fim</code> . . . . .                                  | 28        |
| 3.8      | Código Final Completo . . . . .                                                       | 29        |
| <b>4</b> | <b>Testes e Resultados</b>                                                            | <b>31</b> |
| 4.1      | Compilação do Programa . . . . .                                                      | 31        |
| 4.2      | Teste 1: Lista com Um Único Elemento . . . . .                                        | 31        |
| 4.2.1    | Entrada . . . . .                                                                     | 31        |
| 4.2.2    | Análise dos Resultados . . . . .                                                      | 32        |
| 4.3      | Teste 2: Lista com Dois Elementos . . . . .                                           | 33        |
| 4.3.1    | Entrada . . . . .                                                                     | 33        |
| 4.3.2    | Análise dos Resultados . . . . .                                                      | 33        |
| 4.4      | Teste 3: Lista Extensa . . . . .                                                      | 34        |
| 4.4.1    | Entrada . . . . .                                                                     | 34        |
| 4.4.2    | Análise Matemática Completa . . . . .                                                 | 34        |
| 4.5      | Teste de Entrada Inválida . . . . .                                                   | 35        |
| 4.5.1    | Entrada . . . . .                                                                     | 35        |
| 4.5.2    | Comportamento do Sistema e Análise Técnica . . . . .                                  | 35        |
| <b>5</b> | <b>Análise do Versionamento</b>                                                       | <b>36</b> |
| 5.1      | Estrutura de Branches . . . . .                                                       | 36        |
| 5.2      | Linha Temporal dos Commits . . . . .                                                  | 37        |
| <b>6</b> | <b>Dificuldades Encontradas</b>                                                       | <b>38</b> |
| 6.1      | Conversão de Strings para Double . . . . .                                            | 38        |
| 6.2      | Tratamento de Listas Vazias . . . . .                                                 | 38        |
| 6.3      | Utilização de Branches no Git . . . . .                                               | 38        |
| 6.4      | Integração via Merge . . . . .                                                        | 39        |
| 6.5      | Testes e Validação . . . . .                                                          | 39        |
| <b>7</b> | <b>Conclusões</b>                                                                     | <b>40</b> |
| 7.1      | Síntese do Trabalho Realizado . . . . .                                               | 40        |
| 7.2      | Objectivos Atingidos . . . . .                                                        | 40        |

|     |                                                            |           |
|-----|------------------------------------------------------------|-----------|
| 7.3 | Aprendizagem de Haskell . . . . .                          | 41        |
| 7.4 | Aprendizagem de Git e GitHub . . . . .                     | 41        |
| 7.5 | Importância do Versionamento . . . . .                     | 41        |
| 7.6 | Importância do Linux . . . . .                             | 42        |
| 7.7 | Considerações Finais . . . . .                             | 42        |
|     | <b>Referências Bibliográficas</b>                          | <b>43</b> |
|     | <b>A Código-Fonte Completo</b>                             | <b>44</b> |
|     | <b>B Lista Completa de Comandos Linux e Git Utilizados</b> | <b>46</b> |

# Lista de Figuras

|     |                                                                                                                                                        |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Criação do directório do projecto e inicialização do repositório Git. . .                                                                              | 22 |
| 3.2 | Criação da branch <code>fase1</code> , commits e merge para a branch <code>master</code> , seguidos da criação da branch <code>fase2</code> . . . . .  | 25 |
| 3.3 | Diagrama do fluxo de desenvolvimento com <i>branches</i> e <i>merges</i> (Fast-forward). . . . .                                                       | 26 |
| 3.4 | Implementação da fase final: commit 4, integração da branch <code>fim</code> na <code>master</code> e primeira execução do programa compilado. . . . . | 28 |
| 4.1 | Execução do programa utilizando uma lista contendo um único elemento (valor 2). . . . .                                                                | 32 |
| 4.2 | Execução do sistema utilizando uma lista com dois elementos (2 e 3). .                                                                                 | 33 |
| 4.3 | Execução do sistema utilizando uma lista extensa de 28 elementos para validação robusta do algoritmo. . . . .                                          | 34 |

# Lista de Tabelas

|     |                                                                  |    |
|-----|------------------------------------------------------------------|----|
| 2.1 | Conceitos fundamentais do Git e respectivas definições . . . . . | 20 |
| 3.1 | Comandos de inicialização e respectivas funções . . . . .        | 22 |
| 4.1 | Resultados do Teste 1 – entrada: $\{2\}$ . . . . .               | 32 |
| 4.2 | Resultados do Teste 2 – entrada: $\{2, 3\}$ . . . . .            | 33 |
| 4.3 | Análise matemática completa – Teste 3 . . . . .                  | 34 |
| 5.1 | Análise das branches do projecto . . . . .                       | 36 |
| 5.2 | Linha temporal dos commits do projecto . . . . .                 | 37 |
| B.1 | Comandos Linux e Git utilizados no projecto . . . . .            | 46 |

# Lista de Códigos

|      |                                                                     |    |
|------|---------------------------------------------------------------------|----|
| 3.1  | Comandos de inicialização do ambiente de trabalho . . . . .         | 21 |
| 3.2  | Estrutura inicial – leitura e validação da entrada . . . . .        | 23 |
| 3.3  | Criação da branch fase1 . . . . .                                   | 24 |
| 3.4  | Funções calcularSoma e calcularMedia – branch fase1 . . . . .       | 25 |
| 3.5  | Integração da branch fase1 na branch master . . . . .               | 26 |
| 3.6  | Criação da branch fase2 . . . . .                                   | 26 |
| 3.7  | Funções obterMaior e obterMenor – branch fase2 . . . . .            | 27 |
| 3.8  | Integração da branch fase2 na branch master . . . . .               | 27 |
| 3.9  | Criação da branch fim . . . . .                                     | 28 |
| 3.10 | Função contarElementos – branch fim . . . . .                       | 28 |
| 3.11 | Código final completo do Sistema de Estatística de Listas . . . . . | 29 |
| 4.1  | Compilação do programa com GHC . . . . .                            | 31 |
| A.1  | Código-fonte completo – script.hs . . . . .                         | 44 |

# Capítulo 1

## Introdução

### 1.1 Contextualização

No panorama tecnológico contemporâneo, as competências de programação representam um pilar estruturante da formação de engenheiros em qualquer ramo da área técnica. No contexto específico da Engenharia Mecatrônica, a capacidade de desenvolver software para processamento de dados, controlo de sistemas e automação de processos constitui um diferencial competitivo de elevado valor.

#### 1.1.1 Importância da Programação Funcional

A programação funcional representa um paradigma de desenvolvimento de software que se distingue fundamentalmente da programação imperativa tradicional. Enquanto a programação imperativa descreve *como* um programa deve executar uma sequência de comandos que alteram o estado do sistema, a programação funcional concentra-se em *o que* o programa deve computar, expressando a solução através de funções matemáticas puras.

As vantagens do paradigma funcional incluem a eliminação de efeitos colaterais indesejados (*side effects*), a facilidade de raciocínio formal sobre o código, a maior facilidade de teste e depuração, e a adequação natural ao processamento paralelo. Em contextos de engenharia, onde a correção e a previsibilidade do software são críticas, o paradigma funcional oferece garantias formais que os paradigmas imperativos não conseguem fornecer de forma nativa.

#### 1.1.2 Importância do Haskell

Haskell é uma linguagem de programação funcional pura, com tipagem estática forte e avaliação preguiçosa (*lazy evaluation*). Foi concebida em 1990 por um comité de investigadores com o objetivo de criar uma linguagem funcional padrão para investigação e ensino. Ao longo das décadas seguintes, Haskell evoluiu para uma plataforma robusta, amplamente utilizada tanto na academia como na indústria, em domínios que vão desde

sistemas financeiros até compiladores e verificação formal de software.

A relevância académica do Haskell reside na sua capacidade de promover uma compreensão profunda dos fundamentos matemáticos da computação, nomeadamente teoria de tipos, cálculo lambda e álgebra de tipos. O estudo de Haskell desenvolve nos estudantes um raciocínio computacional mais rigoroso e transferível para qualquer outra linguagem ou paradigma.

### 1.1.3 Importância do Git

O Git é um sistema de controlo de versões distribuído, criado por Linus Torvalds em 2005, originalmente para gerir o desenvolvimento do kernel Linux. Tornou-se o padrão de facto para controlo de versões em projetos de software de qualquer dimensão, sendo utilizado por praticamente todas as organizações tecnológicas do mundo.

O Git permite que os programadores registem o histórico completo das alterações efectuadas num projeto, revertam para versões anteriores em caso de erro, trabalhem em paralelo em diferentes funcionalidades através de ramificações (*branches*), e integrem contribuições de múltiplos colaboradores de forma organizada. No contexto deste projeto, o Git foi utilizado como ferramenta essencial de documentação do processo de desenvolvimento incremental.

### 1.1.4 Importância do Linux no Desenvolvimento de Software

O Linux é um sistema operativo de código aberto, baseado no kernel criado por Linus Torvalds em 1991. Representa o ambiente de desenvolvimento preferencial para a grande maioria dos programadores profissionais, servidores, supercomputadores e sistemas embarcados a nível mundial.

Para o desenvolvimento em Haskell, o Linux oferece um ambiente nativo superior, com instalação simplificada do compilador GHC (*Glasgow Haskell Compiler*), integração directa com o terminal e as ferramentas de desenvolvimento, e ausência de camadas de abstracção que em outros sistemas operativos podem complicar o fluxo de trabalho. O domínio do terminal Linux é, portanto, uma competência indispensável para qualquer engenheiro de software moderno.

## 1.2 Problema

O presente trabalho surge da necessidade de processamento eficiente de listas numéricas para extração de métricas estatísticas fundamentais. Em muitos contextos de engenharia, é necessário analisar conjuntos de dados numéricos para obter rapidamente

informações como a soma total, a média, os valores extremos e a dimensão do conjunto. A implementação de um sistema que automatize este processo, com verificação rigorosa de tipos e comportamento previsível, constitui o problema central que este projeto procura resolver.

## 1.3 Objetivo Geral

Desenvolver um sistema de estatística de listas em Haskell, executado em ambiente Linux, com controlo de versões gerido pelo Git e hospedagem no GitHub, capaz de processar listas de números fornecidas pelo utilizador e calcular automaticamente as métricas estatísticas fundamentais.

## 1.4 Objetivos Específicos

1. Ler e processar listas de números fornecidas pelo utilizador através do terminal Linux;
2. Implementar a função de cálculo da soma de todos os elementos da lista;
3. Implementar a função de cálculo da média aritmética da lista;
4. Implementar a função de determinação do maior valor da lista;
5. Implementar a função de determinação do menor valor da lista;
6. Implementar a função de contagem do número de elementos da lista;
7. Utilizar o Git para controlo e documentação do processo de desenvolvimento;
8. Utilizar o GitHub para hospedagem remota e gestão do repositório;
9. Demonstrar o uso de *branches* para desenvolvimento paralelo e incremental.

## 1.5 Metodologia

O desenvolvimento do projeto seguiu uma metodologia de desenvolvimento incremental baseada no modelo de ramificações do Git. O processo foi organizado em quatro etapas sequenciais, cada uma implementada numa *branch* dedicada, sendo a integração realizada através de operações de *merge* na *branch* principal (**master**).

O fluxo de trabalho adoptado pode ser descrito como: (1) criação da estrutura base na *branch* principal; (2) desenvolvimento incremental de funcionalidades em *branches* dedicadas; (3) integração de cada *branch* na *branch* principal após conclusão e teste; (4) realização de testes manuais com múltiplas entradas para validação do sistema.

Este modelo garante um histórico de commits limpo, onde cada *commit* corresponde a um estado funcional do sistema.

# Capítulo 2

## Fundamentação Teórica

### 2.1 Haskell

#### 2.1.1 História

Haskell foi criada em 1990 como resultado do trabalho do *Haskell Committee*, um grupo de investigadores de universidades norte-americanas e europeias que reconheceram a necessidade de uma linguagem funcional padrão que pudesse servir de base para investigação académica. O nome foi atribuído em homenagem ao lógico Haskell Curry, cujo trabalho em lógica combinatória e cálculo lambda constitui um dos fundamentos matemáticos da programação funcional.

Desde a versão inicial Haskell 1.0 de 1990, a linguagem evoluiu significativamente, tendo o padrão Haskell 98 e posteriormente Haskell 2010 consolidado a especificação moderna da linguagem. O compilador GHC (*Glasgow Haskell Compiler*) tornou-se a implementação de referência, oferecendo desempenho comparável a linguagens compiladas como C e C++ em muitos cenários.

#### 2.1.2 Características Fundamentais

Haskell distingue-se de outras linguagens de programação por um conjunto de características únicas:

- **Pureza funcional:** todas as funções em Haskell são matemática e formalmente puras, no sentido em que a mesma entrada produz sempre o mesmo resultado, sem efeitos secundários no estado do sistema;
- **Tipagem estática forte:** o sistema de tipos de Haskell verifica a correção do programa em tempo de compilação, prevenindo uma vasta classe de erros em tempo de execução;
- **Avaliação preguiçosa (*lazy evaluation*):** as expressões em Haskell só são avaliadas quando os seus resultados são necessários, permitindo a definição de estruturas de dados infinitas e a optimização automática de certas computações;

- **Inferência de tipos:** o compilador é capaz de deduzir automaticamente os tipos da maioria das expressões, reduzindo a necessidade de anotações de tipo explícitas;
- **Polimorfismo paramétrico:** as funções podem operar sobre tipos genéricos, aumentando a reutilização de código.

### 2.1.3 Paradigma Funcional

No paradigma funcional, o programa é conceptualmente uma função matemática que transforma uma entrada numa saída. As funções são tratadas como valores de primeira classe, podendo ser passadas como argumentos, devolvidas como resultados e armazenadas em estruturas de dados. A recursão substitui os ciclos iterativos como mecanismo primário de repetição, e a imutabilidade dos dados elimina uma categoria inteira de erros associados à modificação inadvertida de estado.

Em Haskell, o paradigma funcional é expresso através de sintaxe concisa e expressiva. A correspondência de padrões (*pattern matching*) permite definir funções de forma elegante para diferentes casos, enquanto as compreensões de lista (*list comprehensions*) expressam transformações sobre coleções de dados de forma próxima da notação matemática.

### 2.1.4 Tipagem Forte

O sistema de tipos de Haskell é um dos mais avançados entre as linguagens de programação de uso geral. Baseado no sistema de tipos de Hindley-Milner, com extensões para classes de tipos (*type classes*) e tipos de dados algébricos (*algebraic data types*), o sistema de tipos de Haskell permite expressar invariantes complexos do programa de forma estática, detectando erros em tempo de compilação que em outras linguagens só seriam detectados em execução.

Para o presente projeto, a tipagem forte manifesta-se nas assinaturas de tipo das funções implementadas, como `calcularSoma :: [Double] -> Double`, que especifica que a função aceita uma lista de números de ponto flutuante e devolve um único número de ponto flutuante.

## 2.2 Linux

### 2.2.1 Sistema Operativo

O Linux é um sistema operativo baseado no kernel homónimo, criado por Linus Torvalds em 1991 e distribuído sob a licença GPL (*GNU General Public License*). É um

sistema operativo multitarefa e multiutilizador, suportando simultaneamente múltiplos processos e sessões de utilizador. A sua arquitetura modular, robustez e abertura ao código-fonte tornam-no a plataforma de eleição para servidores, estações de trabalho de desenvolvimento e sistemas embarcados.

### 2.2.2 Terminal e Shell

O terminal é a interface de linha de comando através da qual o utilizador interage directamente com o sistema operativo Linux. A *shell* é o interpretador de comandos que processa as instruções do utilizador, executando programas, manipulando ficheiros e gerindo processos. Entre as *shells* mais comuns em Linux destacam-se o Bash (*Bourne Again Shell*), o Zsh e o Fish.

No contexto deste projeto, o terminal Linux constituiu o ambiente primário de desenvolvimento, compilação e execução do sistema em Haskell, bem como a interface para todos os comandos Git utilizados ao longo do processo.

### 2.2.3 Gestão de Ficheiros

A gestão de ficheiros em Linux é realizada através de um conjunto de comandos disponíveis no terminal. Os comandos fundamentais utilizados neste projeto incluem: `pwd` para verificar o diretório atual, `mkdir` para criar diretórios, `cd` para navegar entre diretórios, `ls` para listar o conteúdo de um diretório, e `touch` para criar ficheiros vazios.

## 2.3 Git

O Git é um sistema de controlo de versões distribuído que regista as alterações efectuadas nos ficheiros de um projeto ao longo do tempo, permitindo que os desenvolvedores revertam para versões anteriores, comparem alterações e colaborem de forma eficiente. A tabela 2.1 apresenta os conceitos fundamentais do Git utilizados neste projeto.

Tabela 2.1: Conceitos fundamentais do Git e respectivas definições

| Conceito            | Definição                                                                                                                                                    |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Repositório</b>  | Directório onde o Git armazena o histórico completo das alterações do projeto, incluindo a pasta oculta <code>.git</code> .                                  |
| <b>Commit</b>       | Instantâneo ( <i>snapshot</i> ) do estado do projeto num determinado momento, identificado por um hash SHA-1 único e acompanhado de uma mensagem descritiva. |
| <b>Branch</b>       | Linha de desenvolvimento independente que permite trabalhar em funcionalidades isoladas sem afectar o código da <i>branch</i> principal.                     |
| <b>Merge</b>        | Operação de integração que combina as alterações de uma <i>branch</i> numa outra, unificando as linhas de desenvolvimento.                                   |
| <b>Checkout</b>     | Comando para navegar entre <i>branches</i> ou restaurar ficheiros para um estado anterior.                                                                   |
| <b>Staging Area</b> | Área intermédia (também designada <i>index</i> ) onde as alterações são preparadas antes de serem incluídas num commit.                                      |
| <b>Fast-forward</b> | Tipo de merge que ocorre quando a <i>branch</i> de destino não divergiu da <i>branch</i> de origem, resultando num avanço linear do ponteiro.                |

## 2.4 GitHub

O GitHub é uma plataforma de hospedagem de repositórios Git na nuvem, que adiciona sobre o Git um conjunto de funcionalidades para colaboração, revisão de código e gestão de projetos. Fundado em 2008 e adquirido pela Microsoft em 2018, o GitHub é actualmente a maior plataforma de hospedagem de código-fonte do mundo, albergando centenas de milhões de repositórios públicos e privados.

No contexto deste projeto, o GitHub serviu como repositório remoto para o alojamento do código-fonte e do README.md, garantindo a persistência e acessibilidade do projeto para além do ambiente local de desenvolvimento.

# Capítulo 3

## Desenvolvimento do Projeto

### 3.1 Criação do Ambiente de Trabalho

O desenvolvimento do projeto teve início com a configuração do ambiente de trabalho em Linux. Os comandos executados no terminal para preparar a estrutura do projeto foram os seguintes:

```
1 # Verificar o directorio de trabalho actual
2 pwd
3
4 # Criar o directorio do projecto
5 mkdir SisEstatistica
6
7 # Navegar para o directorio criado
8 cd SisEstatistica
9
10 # Inicializar o repositorio Git local
11 git init
12
13 # Criar os ficheiros iniciais do projecto
14 touch script.hs README.md
```

Código 3.1: Comandos de inicialização do ambiente de trabalho

A tabela [3.1](#) descreve a função de cada comando executado nesta fase.

Tabela 3.1: Comandos de inicialização e respectivas funções

| Comando                                    | Função                                                                                                                                       |
|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>pwd</code>                           | Imprime o caminho absoluto do directório de trabalho actual, permitindo confirmar a localização no sistema de ficheiros.                     |
| <code>mkdir<br/>SisEstatistica</code>      | Cria um novo directório denominado <code>SisEstatistica</code> no local actual, que servirá como pasta raiz do projecto.                     |
| <code>cd SisEstatistica</code>             | Altera o directório de trabalho actual para o directório <code>SisEstatistica</code> recém-criado.                                           |
| <code>ls</code>                            | Lista os ficheiros e directórios contidos no directório actual, confirmando a estrutura criada.                                              |
| <code>git init</code>                      | Inicializa um repositório Git vazio no directório actual, criando a pasta oculta <code>.git</code> que armazena todo o histórico de versões. |
| <code>touch script.hs<br/>README.md</code> | Cria dois ficheiros vazios: <code>script.hs</code> (ficheiro de código Haskell) e <code>README.md</code> (documentação do projecto).         |

```

> pwd
/home/edineladelso/starb/dev/github
> mkdir SisEstatistica
> cd SisEstatistica
> ls
> git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Repositório vazio Git inicializado em /home/edineladelso/starb/dev/github/SisEstatistica/.git/
> touch script.hs README.md
> code script.hs
> code .

~/starb/dev/github/SisEstatistica | on master ?2
> git add .
> git commit -m "commit 1: Estrutura inicial e captura de entrada do utilizador"
[master (root-commit) 9326402] commit 1: Estrutura inicial e captura de entrada do utilizador
2 files changed, 27 insertions(+)
create mode 100644 README.md
create mode 100644 script.hs
> git checkout -b fase1
Switched to a new branch 'fase1'
> code .

```

Figura 3.1: Criação do directório do projecto e inicialização do repositório Git.

A figura 3.1 mostra o terminal Linux após a execução dos comandos de iniciali-

zação. É possível observar: a confirmação do directório de trabalho pelo comando `pwd` (`/home/edineladelso/starb/dev/github`); a criação do directório `SisEstatistica`; a navegação para o interior do directório com `cd`; a execução de `git init` que inicializou o repositório vazio, tendo o Git emitido uma mensagem de aviso (*hint*) a indicar que o nome da *branch* inicial é `master` e que este comportamento pode ser reconfigurado; a criação dos ficheiros `script.hs` e `README.md` com o comando `touch`; e finalmente a execução dos comandos `git add .` e `git commit` que registaram o primeiro *commit* do projecto com a mensagem "*commit 1: Estrutura inicial e captura de entrada do utilizador*", correspondendo à criação de 2 ficheiros com 27 inserções. Na parte inferior da figura é também visível a criação da *branch fase1* com o comando `git checkout -b fase1` e a confirmação da mudança para a nova *branch*.

## 3.2 Implementação da Estrutura Inicial

Na *branch master*, foi implementada a estrutura base do programa em Haskell. Esta estrutura é responsável pela captura da entrada do utilizador, pela conversão da *string* de entrada numa lista de números do tipo `Double` e pela validação básica da entrada.

O Código 3.2 apresenta o conteúdo do ficheiro `script.hs` após o primeiro *commit*.

```

1  -- Ficheiro: script.hs
2  -- Sistema de Estatística de Listas em Haskell
3  -- Grupo 4
4
5  module Main where
6
7  -- Funcao principal que inicia a execucao do programa
8  main :: IO ()
9  main = do
10     putStrLn "===== "
11     putStrLn "Sistema de Estatistica de listas"
12     putStrLn "===== "
13     entrada <- getLine
14
15     -- Converte a string de entrada numa lista de Double
16     let listaNumeros = map read (words entrada) :: [Double]
17
18     putStrLn "===== RESULTADOS ===== "
19

```

```
20     if null listaNumeros
21     then putStrLn "Erro: A lista nao pode estar vazia!"
22     else do
23         putStrLn "\n--- Resultados ---"
24         putStrLn ("Lista processada com sucesso: "
25                 ++ show listaNumeros)
```

Código 3.2: Estrutura inicial – leitura e validação da entrada

Analisando o código linha por linha: a declaração `module Main where` define o módulo principal do programa; a assinatura `main :: IO ()` especifica que a função `main` é uma acção de entrada/saída (IO) que não devolve valor; `putStrLn` imprime uma linha de texto no terminal; `getLine` lê uma linha de entrada do utilizador e armazena-a na variável `entrada`; a expressão `words entrada` divide a *string* de entrada em palavras usando os espaços como separadores; `map read` aplica a função `read` a cada palavra, convertendo-a para o tipo `Double`; a anotação `:: [Double]` especifica explicitamente o tipo da lista resultante; `null listaNumeros` verifica se a lista está vazia; e o bloco do com indentação dentro do `else` executa as acções sequencialmente quando a lista contém elementos.

## 3.3 Desenvolvimento da Branch fase1

Após o primeiro *commit* na *branch* `master`, o desenvolvimento prosseguiu com a criação da *branch* `fase1`, dedicada à implementação das funções de cálculo da soma e da média.

### 3.3.1 Criação da Branch e Conceito de Ramificação

O comando utilizado para criar e mudar simultaneamente para a nova *branch* foi:

```
1 git checkout -b fase1
```

Código 3.3: Criação da branch fase1

O parâmetro `-b` instrui o Git a criar uma nova *branch* com o nome especificado (`fase1`) e a mudar imediatamente para ela. Uma *branch* no Git é essencialmente um ponteiro leve para um *commit* específico, que avança automaticamente a cada novo *commit* realizado nessa *branch*. A criação de uma *branch* para cada fase de desenvolvimento garante que as alterações em curso não interferem com o código estável da *branch* principal, permitindo trabalhar de forma isolada e segura.

### 3.3.2 Implementação de calcularSoma e calcularMedia

Na *branch* `fase1`, foram implementadas as funções `calcularSoma` e `calcularMedia`:

```

1  -- Funcao que calcula a soma de todos os elementos
2  calcularSoma :: [Double] -> Double
3  calcularSoma [] = 0
4  calcularSoma xs = sum xs
5
6  -- Funcao que calcula a media aritmetica
7  calcularMedia :: [Double] -> Double
8  calcularMedia xs = calcularSoma xs / fromIntegral (length xs)

```

Código 3.4: Funções `calcularSoma` e `calcularMedia` – *branch* `fase1`

A função `calcularSoma` utiliza correspondência de padrões (*pattern matching*): quando recebe uma lista vazia `[]`, devolve 0; para qualquer outra lista `xs`, aplica a função nativa do Haskell `sum`, que calcula a soma de todos os elementos. A função `calcularMedia` divide a soma total (obtida por `calcularSoma xs`) pelo número de elementos da lista. A conversão `fromIntegral` é necessária porque `length` devolve um valor do tipo `Int`, enquanto a divisão requer ambos os operandos do tipo `Double`.

```

> code
> git add script.hs
> git commit -m "commit 2: Implementacao de funcoes calcularSoma e CalcularMedia"
[fase1 aed2c6f] commit 2: Implementacao de funcoes calcularSoma e CalcularMedia
1 file changed, 15 insertions(+), 1 deletion(-)
> git checkout master
Switched to branch 'master'
> git merge fase1
Updating 9326402..aed2c6f
Fast-forward
 script.hs | 16 ++++++
1 file changed, 15 insertions(+), 1 deletion(-)
> git checkout -b fase2
Switched to a new branch 'fase2'
> code
> git add script.hs
> git commit -m "commit 3: Adicionadas as funcoes obterMaior e obterMenor"
[fase2 e2f6fe2] commit 3: Adicionadas as funcoes obterMaior e obterMenor
1 file changed, 10 insertions(+)
> git checkout master
Switched to branch 'master'
> git merge fase2
Updating aed2c6f..e2f6fe2
Fast-forward
 script.hs | 10 ++++++
1 file changed, 10 insertions(+)
> git checkout -b fim
Switched to a new branch 'fim'
> code
> git add script.hs
> git commit -m "Commit 4: Implementação da contagem de elementos e interface final"

```

Figura 3.2: Criação da *branch* `fase1`, commits e merge para a *branch* `master`, seguidos da criação da *branch* `fase2`.

A figura 3.2 documenta o fluxo completo de desenvolvimento das *branches* `fase1` e `fase2`. Na parte superior da imagem, observa-se o *commit* 2 na *branch* `fase1` com a mensagem "*commit 2: Implementacao de funcoes calcularSoma e CalcularMedia*",

indicando que 1 ficheiro foi alterado com 15 inserções e 1 remoção. De seguida, o regresso à *branch* `master` com `git checkout master` e a integração via `git merge fase1`, que produziu um *merge* do tipo *Fast-forward*, actualizando o ponteiro de 9326402 para aed2c6f.

### 3.4 Integração da Branch `fase1` na Branch Principal

Após a conclusão e teste das funções implementadas na *branch* `fase1`, procedeu-se à sua integração na *branch* `master`:

```
1 git checkout master
2 git merge fase1
```

Código 3.5: Integração da branch `fase1` na branch `master`

O resultado do *merge* foi do tipo *Fast-forward*, o que significa que a *branch* `master` não continha *commits* novos desde a criação da *branch* `fase1`. Neste caso, o Git simplesmente avança o ponteiro da *branch* `master` para apontar para o *commit* mais recente da *branch* `fase1`, sem criar um *commit* de *merge* adicional.

A figura 3.3 ilustra graficamente o fluxo de desenvolvimento com *branches* e *merges* utilizado neste projeto.

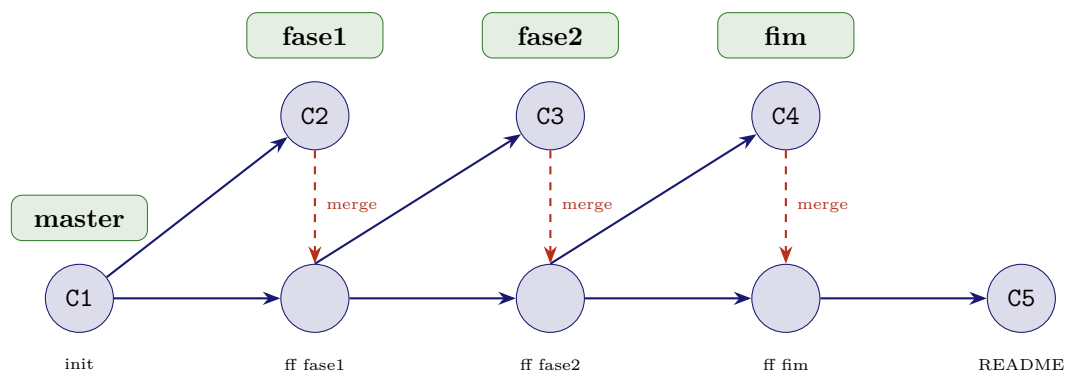


Figura 3.3: Diagrama do fluxo de desenvolvimento com *branches* e *merges* (Fast-forward).

### 3.5 Desenvolvimento da Branch `fase2`

Com a *branch* `fase1` integrada com sucesso na *branch* `master`, prosseguiu-se com a criação da *branch* `fase2` para implementação das funções de determinação do maior e do menor valor.

```
1 git checkout -b fase2
```

---

### Código 3.6: Criação da branch fase2

Conforme visível na figura 3.2, o Git confirmou a criação e mudança para a nova *branch* com a mensagem *"Switched to a new branch 'fase2'"*.

#### 3.5.1 Implementação das Funções obterMaior e obterMenor

As funções obterMaior e obterMenor foram implementadas recorrendo às funções nativas do Prelude do Haskell:

```
1  -- Funcao para encontrar o maior valor da lista
2  obterMaior :: [Double] -> Double
3  obterMaior xs = maximum xs
4
5  -- Funcao para encontrar o menor valor da lista
6  obterMenor :: [Double] -> Double
7  obterMenor xs = minimum xs
```

### Código 3.7: Funções obterMaior e obterMenor – branch fase2

A função `maximum` percorre a lista e devolve o elemento de maior valor, utilizando a relação de ordem definida para o tipo `Double`. De forma análoga, `minimum` devolve o elemento de menor valor. Ambas as funções são polimórficas e operam sobre qualquer tipo que implemente a classe de tipos `Ord` (tipos ordenáveis).

O *commit* 3 na *branch* `fase2` foi registado com a mensagem *"commit 3: Adicionadas as funcoes obterMaior e obterMenor"*, com 1 ficheiro modificado e 10 inserções, conforme visível na figura 3.2. A integração na *branch* `master` foi realizada via `git merge fase2`, resultando novamente num *merge* do tipo *Fast-forward*, actualizando o ponteiro de `aed2c6f` para `e2f6fe2`.

## 3.6 Integração da Branch fase2

```
1  git checkout master
2  git merge fase2
```

### Código 3.8: Integração da branch fase2 na branch master

### 3.7 Desenvolvimento da Branch fim

A fase final do desenvolvimento correspondeu à criação da *branch* `fim`, na qual foi implementada a função de contagem de elementos e finalizada a interface do programa.

```
1 git checkout -b fim
```

Código 3.9: Criação da branch `fim`

```
1 -- Funcao para contar a quantidade de elementos
2 contarElementos :: [Double] -> Int
3 contarElementos xs = length xs
```

Código 3.10: Função `contarElementos` – branch `fim`

A função `length` do Prelude do Haskell devolve o número de elementos de uma lista como um valor do tipo `Int`. A sua utilização é directa e eficiente para a maioria dos casos práticos.

```
> git add script.hs
> git commit -m "Commit 4: Implementação da contagem de elementos e interface final"

! Concluir a integração de todas as ramificações na branch main
git checkout master
git merge fim
[fim cc39b9d] Commit 4: Implementação da contagem de elementos e interface final
 1 file changed, 10 insertions(+), 6 deletions(-)
Switched to branch 'master'
Updating e2f6fe2..cc39b9d
Fast-forward
 script.hs | 16 ++++++-----
 1 file changed, 10 insertions(+), 6 deletions(-)
> ghc -o programa script.hs
[1 of 1] Compiling Main             ( script.hs, script.o )
Linking programa ...
> ./programa
=====
Sistema de Estatistica de listas
2
=====  RESULTADOS  =====
--- Resultados ---
0. Lista de numeros: [2.0]
1. Soma: 2.0
2. Média: 2.0
3. Maior Valor: 2.0
4. Menor Valor: 2.0
5. Quantidade de elementos : 1
> ./programa
```

Figura 3.4: Implementação da fase final: commit 4, integração da branch `fim` na `master` e primeira execução do programa compilado.

A figura 3.4 documenta a fase final do desenvolvimento. Observam-se os seguintes passos: o *commit* 4 com a mensagem "*Commit 4: Implementação da contagem de elementos e interface final*"; o regresso à *branch* `master` com `git checkout master`; o *merge* da *branch* `fim` (`git merge fim`), que produziu novamente um *Fast-forward* de `e2f6fe2` para `cc39b9d`; a compilação do programa com o comando `ghc -o programa script.hs`, que produziu o executável `programa` a partir do módulo `Main` em `script.hs`; e a primeira execução do programa compilado com `./programa`, introduzindo o valor 2 e verificando que os resultados (Soma: 2.0, Média: 2.0, Maior Valor: 2.0, Menor Valor: 2.0, Quantidade: 1) são correctos.

## 3.8 Código Final Completo

O código final completo do Sistema de Estatística de Listas, resultante da integração de todas as *branches*, é apresentado no Código 3.11.

```
1  -- Ficheiro: script.hs
2  -- Sistema de Estatística de Listas em Haskell
3  -- Grupo 4 -- Licenciatura em Engenharia Mecatronica
4
5  module Main where
6
7  -- Funcao que calcula a soma de todos os elementos
8  calcularSoma :: [Double] -> Double
9  calcularSoma [] = 0
10 calcularSoma xs = sum xs
11
12 -- Funcao que calcula a media aritmetica
13 calcularMedia :: [Double] -> Double
14 calcularMedia xs = calcularSoma xs / fromIntegral (length xs)
15
16 -- Funcao para encontrar o maior valor da lista
17 obterMaior :: [Double] -> Double
18 obterMaior xs = maximum xs
19
20 -- Funcao para encontrar o menor valor da lista
21 obterMenor :: [Double] -> Double
22 obterMenor xs = minimum xs
23
24 -- Funcao para contar a quantidade de elementos
25 contarElementos :: [Double] -> Int
```

```
26 contarElementos xs = length xs
27
28 -- Funcao principal do programa
29 main :: IO ()
30 main = do
31     putStrLn "===== "
32     putStrLn "Sistema de Estatistica de listas"
33     putStrLn "===== "
34     entrada <- getLine
35
36     let listaNumeros = map read (words entrada) :: [Double]
37
38     putStrLn "===== RESULTADOS ===== "
39
40     if null listaNumeros
41     then putStrLn "Erro: A lista nao pode estar vazia!"
42     else do
43         putStrLn "\n--- Resultados ---"
44         putStrLn ("0. Lista de numeros: "
45             ++ show listaNumeros)
46         putStrLn ("1. Soma: "
47             ++ show (calcularSoma listaNumeros))
48         putStrLn ("2. Media: "
49             ++ show (calcularMedia listaNumeros))
50         putStrLn ("3. Maior Valor: "
51             ++ show (obterMaior listaNumeros))
52         putStrLn ("4. Menor Valor: "
53             ++ show (obterMenor listaNumeros))
54         putStrLn ("5. Quantidade de elementos : "
55             ++ show (contarElementos listaNumeros))
```

Código 3.11: Código final completo do Sistema de Estatística de Listas

# Capítulo 4

## Testes e Resultados

A fase de testes constitui uma etapa fundamental do processo de desenvolvimento de software, permitindo validar a correção dos algoritmos implementados e identificar comportamentos inesperados. Neste capítulo são apresentados e analisados os resultados dos testes realizados com diferentes entradas.

### 4.1 Compilação do Programa

Antes da realização dos testes, o código Haskell foi compilado utilizando o GHC (*Glasgow Haskell Compiler*):

```
1 ghc -o programa script.hs
```

Código 4.1: Compilação do programa com GHC

O GHC compilou o módulo `Main` do ficheiro `script.hs`, gerou o ficheiro objecto `script.o` e ligou (*linked*) os módulos para produzir o executável `programa`. A compilação foi concluída sem erros, confirmando a correção sintáctica e semântica do código.

### 4.2 Teste 1: Lista com Um Único Elemento

#### 4.2.1 Entrada

O primeiro teste foi realizado com uma lista contendo apenas um elemento: o valor 2.

```

> ./programa
=====
Sistema de Estatística de listas
ww 22 3 2 1

=====  RESULTADOS  =====

--- Resultados ---
programa: Prelude.read: no parse
> ./programa
=====
Sistema de Estatística de listas
2 3

=====  RESULTADOS  =====

--- Resultados ---
0. Lista de numeros: [2.0,3.0]
1. Soma: 5.0
2. Média: 2.5
3. Maior Valor: 3.0
4. Menor Valor: 2.0
5. Quantidade de elementos : 2

```

Figura 4.1: Execução do programa utilizando uma lista contendo um único elemento (valor 2).

## 4.2.2 Análise dos Resultados

A figura 4.1 apresenta a saída do programa para a entrada 2. Os resultados obtidos foram:

Tabela 4.1: Resultados do Teste 1 – entrada: {2}

| Métrica     | Resultado Obtido | Verificação Matemática |
|-------------|------------------|------------------------|
| Lista       | [2.0]            | $\{2\}$                |
| Soma        | 2.0              | $\sum = 2$             |
| Média       | 2.0              | $\bar{x} = 2/1 = 2$    |
| Maior Valor | 2.0              | $\max\{2\} = 2$        |
| Menor Valor | 2.0              | $\min\{2\} = 2$        |
| Quantidade  | 1                | $ \{2\}  = 1$          |

Todos os resultados são matematicamente correctos. Para uma lista com um único elemento, é esperado que a soma, a média, o maior valor e o menor valor sejam todos iguais ao único elemento da lista. A quantidade é correctamente reportada como 1.

### 4.3 Teste 2: Lista com Dois Elementos

#### 4.3.1 Entrada

O segundo teste utilizou a lista com dois elementos: 2 3.

```
> ./programa
=====
Sistema de Estatística de Listas
1 2 3 4 5 5 6 7 4 6 7 7 3 5 5 7 10 4 33 22 11 55 66 77 88 99 00
=====
RESULTADOS
=====
--- Resultados ---
0. Lista de numeros: [1.0,2.0,3.0,4.0,5.0,5.0,6.0,6.0,7.0,4.0,6.0,7.0,7.0,3.0,5.0,5.0,7.0,10.0,4.0,33.0,22.0,11.0,55.0,66.0,77.0,88.0,99.0,0.0]
1. Soma: 548.0
2. Média: 19.571428571428573
3. Maior Valor: 99.0
4. Menor Valor: 0.0
5. Quantidade de elementos : 28
> ./programa
=====
Sistema de Estatística de Listas
ww 22 3 2 1
=====
RESULTADOS
=====
--- Resultados ---
programa: Prelude.read: no parse
> ./programa
=====
Sistema de Estatística de Listas
2 3
=====
RESULTADOS
=====
--- Resultados ---
0. Lista de numeros: [2.0,3.0]
```

Figura 4.2: Execução do sistema utilizando uma lista com dois elementos (2 e 3).

#### 4.3.2 Análise dos Resultados

A figura 4.2 apresenta os resultados para a entrada 2 3. A tabela 4.2 valida matematicamente cada resultado obtido.

Tabela 4.2: Resultados do Teste 2 – entrada: {2, 3}

| Métrica     | Resultado Obtido | Verificação Matemática |
|-------------|------------------|------------------------|
| Lista       | [2.0, 3.0]       | {2, 3}                 |
| Soma        | 5.0              | $2 + 3 = 5$            |
| Média       | 2.5              | $5/2 = 2.5$            |
| Maior Valor | 3.0              | $\max\{2, 3\} = 3$     |
| Menor Valor | 2.0              | $\min\{2, 3\} = 2$     |
| Quantidade  | 2                | $ \{2, 3\}  = 2$       |

Todos os resultados estão matematicamente correctos. O teste valida o funcionamento correcto das cinco funções implementadas para uma lista com dois elementos distintos.

## 4.4 Teste 3: Lista Extensa

### 4.4.1 Entrada

O terceiro teste foi realizado com uma lista extensa de 28 elementos:

1 2 3 4 5 5 6 6 7 4 6 7 7 3 5 5 7 10 4 33 22 11 55 66 77 88 99 00

```

> git commit -m "Commit 4: Implementação da contagem de elementos e interface final"
[cc39b9d] Commit 4: Implementação da contagem de elementos e interface final
1 file changed, 10 insertions(+), 6 deletions(-)
Switched to branch 'master'
Updating e2f6fe2..cc39b9d
Fast-forward
 script.hs | 16 ++++++-----
 1 file changed, 10 insertions(+), 6 deletions(-)
> ghc -o programa script.hs
[1 of 1] Compiling Main             ( script.hs, script.o )
Linking programa ...
> ./programa
=====
Sistema de Estatística de listas
2
=====  RESULTADOS  =====
--- Resultados ---
0. Lista de numeros: [2.0]
1. Soma: 2.0
2. Média: 2.0
3. Maior Valor: 2.0
4. Menor Valor: 2.0
5. Quantidade de elementos : 1
> ./programa
=====

```

Figura 4.3: Execução do sistema utilizando uma lista extensa de 28 elementos para validação robusta do algoritmo.

### 4.4.2 Análise Matemática Completa

Tabela 4.3: Análise matemática completa – Teste 3

| Métrica     | Resultado do Sistema | Verificação Matemática              |
|-------------|----------------------|-------------------------------------|
| Quantidade  | 28                   | 28 elementos contados               |
| Soma        | 548.0                | Soma de todos os 28 elementos = 548 |
| Média       | 19.571...            | $548 \div 28 \approx 19,5714$       |
| Maior Valor | 99.0                 | O maior elemento da lista é 99      |
| Menor Valor | 0.0                  | O valor 00 é interpretado como 0    |

A média calculada pelo sistema, 19.571428571428573, corresponde com precisão ao valor matemático exacto  $548/28 = 19.\overline{571428}$ . O valor 00 introduzido pelo utilizador é correctamente interpretado pelo Haskell como o número 0, sendo reportado como o menor valor da lista.

## 4.5 Teste de Entrada Inválida

### 4.5.1 Entrada

O quarto teste foi realizado com uma entrada contendo uma sequência de caracteres inválida: `ww 22 3 2 1`, onde `ww` não pode ser convertido para um número do tipo `Double`.

### 4.5.2 Comportamento do Sistema e Análise Técnica

O sistema reportou o erro: `programa: Prelude.read: no parse`. Este comportamento é resultado do funcionamento da função `read` do `Prelude` do Haskell, que tenta converter uma *string* para um tipo especificado. Quando a conversão não é possível (como é o caso de `"ww"` para o tipo `Double`), a função `read` lança uma exceção em tempo de execução com a mensagem `Prelude.read: no parse`.

A expressão `map read (words entrada) :: [Double]` processa a lista de palavras de forma sequencial. Ao tentar converter `"ww"` para `Double`, a função `read` falha imediatamente e a exceção interrompe a execução do programa antes de qualquer cálculo ser efectuado.

Este comportamento, embora não tratado de forma elegante nesta versão do programa, é tecnicamente previsível e documentado. Uma versão mais robusta do sistema poderia utilizar a função `readMaybe` do módulo `Text.Read` para tratar este caso de forma segura, devolvendo um valor `Nothing` para entradas inválidas em vez de lançar uma exceção.

# Capítulo 5

## Análise do Versionamento

### 5.1 Estrutura de Branches

A tabela 5.1 apresenta a estrutura completa de *branches* utilizada no projecto e o resultado de cada fase de desenvolvimento.

Tabela 5.1: Análise das branches do projecto

| Branch              | Objetivo                                                                                 | Resultado                                                                                                                                                                         |
|---------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>master</code> | Ramo principal. Estrutura base e captura de entrada do utilizador.                       | Ficheiros iniciais criados ( <code>script.hs</code> , <code>README.md</code> ). Primeiro <i>commit</i> registado com o hash 9326402.                                              |
| <code>fase1</code>  | Implementação das funções de cálculo da soma e da média aritmética.                      | Funções <code>calcularSoma</code> e <code>calcularMedia</code> implementadas e testadas. Commit <code>aed2c6f</code> . Integrada na <code>master</code> via <i>fast-forward</i> . |
| <code>fase2</code>  | Implementação das funções de determinação do maior e do menor valor da lista.            | Funções <code>obterMaior</code> e <code>obterMenor</code> implementadas. Commit <code>e2f6fe2</code> . Integrada na <code>master</code> via <i>fast-forward</i> .                 |
| <code>fim</code>    | Implementação da função de contagem de elementos e finalização da interface do programa. | Função <code>contarElementos</code> implementada. Interface final definida. Commit <code>cc39b9d</code> . Integrada na <code>master</code> via <i>fast-forward</i> .              |

## 5.2 Linha Temporal dos Commits

A tabela 5.2 apresenta a cronologia completa dos *commits* realizados ao longo do desenvolvimento do projecto.

Tabela 5.2: Linha temporal dos commits do projecto

| # | Branch | Descrição do Commit                                                                                                                                                                                        |
|---|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | master | Estrutura inicial e captura de entrada do utilizador. Criação dos ficheiros <code>script.hs</code> e <code>README.md</code> com a estrutura base do programa e a lógica de leitura e conversão da entrada. |
| 2 | fase1  | Implementação das funções <code>calcularSoma</code> e <code>calcularMedia</code> . Adição das duas primeiras métricas estatísticas e actualização da função <code>main</code> para exibir os resultados.   |
| 3 | fase2  | Adição das funções <code>obterMaior</code> e <code>obterMenor</code> . Extensão do sistema com as funções de determinação dos valores extremos da lista.                                                   |
| 4 | fim    | Implementação da contagem de elementos e interface final. Adição da função <code>contarElementos</code> e refinamento da interface de saída do programa.                                                   |
| 5 | master | Actualização completa do ficheiro <code>README.md</code> com documentação do projecto, instruções de compilação e execução, e descrição das funcionalidades.                                               |

# Capítulo 6

## Dificuldades Encontradas

### 6.1 Conversão de Strings para Double

Uma das principais dificuldades encontradas durante o desenvolvimento foi a correcta conversão dos dados de entrada, fornecidos pelo utilizador como uma *string*, para valores numéricos do tipo `Double`. Em Haskell, esta conversão exige a combinação das funções `words` (para separar a *string* em palavras), `map` (para aplicar uma função a cada elemento da lista) e `read` (para converter cada palavra para o tipo desejado), bem como a anotação de tipo explícita `:: [Double]` para informar o compilador do tipo pretendido.

A necessidade da conversão `fromIntegral` na função `calcularMedia`, para compatibilizar o resultado inteiro de `length` com o tipo `Double` da divisão, foi igualmente uma fonte de confusão inicial, ilustrando a rigidez do sistema de tipos de Haskell.

### 6.2 Tratamento de Listas Vazias

O tratamento de listas vazias constituiu outro desafio técnico relevante. As funções `maximum` e `minimum` do Prelude do Haskell lançam uma excepção quando aplicadas a uma lista vazia. Por isso, foi necessário implementar uma verificação prévia com a função `null` na função `main`, de forma a garantir que as funções de cálculo não são invocadas com uma lista vazia.

### 6.3 Utilização de Branches no Git

A adopção de um modelo de desenvolvimento baseado em *branches* exigiu uma compreensão clara dos conceitos de criação de *branches*, mudança entre *branches* e integração via *merge*. A principal dificuldade residiu em compreender a diferença entre um *merge* do tipo *fast-forward* e um *merge* com criação de *commit* adicional, bem como as situações em que cada tipo de *merge* ocorre.

## 6.4 Integração via Merge

A sequência correcta de comandos para integrar uma *branch* na *branch* principal exige que o utilizador regresse primeiro à *branch* de destino (`git checkout master`) antes de executar o *merge* (`git merge nome-da-branch`). A realização do *merge* na *branch* errada poderia ter consequências negativas para o histórico do projecto.

## 6.5 Testes e Validação

A definição de casos de teste representativos, incluindo casos extremos como listas com um único elemento e entradas inválidas, exigiu um esforço de análise e criatividade por parte do grupo. O comportamento do sistema perante entradas inválidas, manifestado pela excepção `Prelude.read: no parse`, revelou a importância de implementar tratamento robusto de erros em aplicações de produção.

# Capítulo 7

## Conclusões

### 7.1 Síntese do Trabalho Realizado

O presente trabalho teve como objectivo o desenvolvimento de um Sistema de Estatística de Listas em Haskell, utilizando o ambiente Linux, o sistema de controlo de versões Git e a plataforma GitHub para hospedagem remota do código. O sistema foi desenvolvido com sucesso, implementando todas as funcionalidades propostas: cálculo da soma, da média, do maior valor, do menor valor e da quantidade de elementos de uma lista numérica fornecida pelo utilizador.

O desenvolvimento seguiu uma metodologia de desenvolvimento incremental baseada em *branches*, com quatro fases claramente delimitadas e documentadas. Cada fase foi registada num *commit* Git com mensagem descritiva, produzindo um histórico de versões limpo e rastreável. As integrações entre *branches* foram realizadas com sucesso através de operações de *merge* do tipo *fast-forward*, o que reflecte a organização sequencial e não divergente do processo de desenvolvimento adoptado.

### 7.2 Objectivos Atingidos

Todos os objectivos específicos definidos na secção de introdução foram alcançados. A função `calcularSoma` calcula correctamente a soma de todos os elementos da lista. A função `calcularMedia` determina a média aritmética com precisão de ponto flutuante. As funções `obterMaior` e `obterMenor` identificam correctamente os valores extremos da lista. A função `contarElementos` reporta com precisão o número de elementos. Os três testes principais com listas de dimensões variadas (1, 2 e 28 elementos) confirmaram a correção dos algoritmos em todos os casos.

A utilização do Git e do GitHub foi demonstrada através de um histórico de 5 *commits* distribuídos por 4 *branches*, documentando rigorosamente o processo de desenvolvimento incremental. O README.md foi actualizado com instruções completas de utilização do projecto.

## 7.3 Aprendizagem de Haskell

O desenvolvimento deste projecto proporcionou ao Grupo 4 uma primeira experiência prática significativa com a linguagem Haskell e o paradigma de programação funcional. A compreensão dos conceitos de correspondência de padrões (*pattern matching*), tipagem estática forte, funções de ordem superior (`map`), e a distinção entre tipos `Int` e `Double` foi aprofundada através da prática efectiva.

A experiência revelou que Haskell, apesar de exigir uma curva de aprendizagem mais íngreme do que as linguagens imperativas convencionais, oferece uma forma de expressão de algoritmos que é simultaneamente concisa e matematicamente rigorosa. O código produzido é compacto, legível e directamente mapeável para as definições matemáticas das operações estatísticas implementadas.

## 7.4 Aprendizagem de Git e GitHub

A utilização prática do Git ao longo deste projecto consolidou a compreensão de um conjunto essencial de operações de controlo de versões: `git init`, `git add`, `git commit`, `git checkout -b`, `git merge`. A observação directa dos hashes de *commit* e das mensagens de *fast-forward* no terminal permitiu compreender o funcionamento interno do sistema de referências do Git de uma forma que a leitura teórica não poderia proporcionar.

O GitHub, como plataforma de hospedagem remota, demonstrou a sua utilidade para a partilha e preservação do código. A actualização do README.md com documentação do projecto é uma prática profissional de valor elevado, que facilita a compreensão do projecto por parte de terceiros e serve como referência rápida para os próprios membros da equipa.

## 7.5 Importância do Versionamento

Este projecto demonstrou de forma concreta a importância do controlo de versões no desenvolvimento de software. A capacidade de registar o histórico completo das alterações, de trabalhar em funcionalidades isoladas sem risco de comprometer o código estável, e de integrar contribuições de forma organizada são vantagens que se manifestam mesmo em projectos de pequena dimensão como este. Em projectos de maior envergadura, com múltiplos colaboradores e funcionalidades complexas, estas vantagens tornam-se absolutamente indispensáveis.

O modelo de *branches* adoptado neste projecto, apesar de simplificado rela-

tivamente aos modelos profissionais como o Git Flow ou o GitHub Flow, ilustra os princípios fundamentais do desenvolvimento paralelo e da integração contínua.

## 7.6 Importância do Linux

O ambiente Linux revelou-se o contexto ideal para o desenvolvimento deste projecto. A disponibilidade nativa do GHC (*Glasgow Haskell Compiler*), a integração directa entre o terminal e as ferramentas de desenvolvimento, e a ausência de fricção no fluxo de trabalho entre o editor de código, o compilador, o Git e o terminal contribuíram para uma experiência de desenvolvimento fluída e eficiente.

O domínio dos comandos fundamentais do terminal Linux (`pwd`, `mkdir`, `cd`, `ls`, `touch`) é uma competência transversal de elevado valor para qualquer profissional de Engenharia Mecatrónica que trabalhe com sistemas de controlo, automação ou processamento de dados, domínios onde o Linux é onnipresente.

## 7.7 Considerações Finais

O Sistema de Estatística de Listas desenvolvido constitui um exemplo bem-sucedido de aplicação integrada de múltiplas competências técnicas: programação funcional em Haskell, utilização profissional do terminal Linux e adopção de boas práticas de controlo de versões com Git e GitHub. O Grupo 4 considera que os objectivos académicos e técnicos do projecto foram plenamente alcançados, e que as competências adquiridas serão de utilidade directa em contextos profissionais futuros.

# Referências Bibliográficas

- [1] B. O’Sullivan, J. Goerzen, and D. Stewart, *Real World Haskell*. Sebastopol, CA, USA: O’Reilly Media, 2009.
- [2] G. Hutton, *Programming in Haskell*, 2nd ed. Cambridge, UK: Cambridge University Press, 2016.
- [3] The GHC Team, *The Glorious Glasgow Haskell Compilation System User’s Guide*, Version 9.x. Glasgow, UK: University of Glasgow, 2024. [Online]. Available: [https://downloads.haskell.org/ghc/latest/docs/users\\_guide/](https://downloads.haskell.org/ghc/latest/docs/users_guide/)
- [4] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014. [Online]. Available: <https://git-scm.com/book/en/v2>
- [5] Linux Foundation, *Introduction to Linux*. San Francisco, CA, USA: The Linux Foundation, 2023. [Online]. Available: <https://www.linuxfoundation.org/>
- [6] GitHub, Inc., *GitHub Documentation*. San Francisco, CA, USA: GitHub, Inc., 2024. [Online]. Available: <https://docs.github.com/>
- [7] P. Hudak, J. Hughes, S. Peyton Jones, and P. Wadler, “A History of Haskell: Being Lazy with Class,” in *Proc. 3rd ACM SIGPLAN Conf. on History of Programming Languages (HOPL-III)*, San Diego, CA, USA, Jun. 2007, pp. 12-1–12-55.
- [8] W. Shotts, *The Linux Command Line*, 5th ed. San Francisco, CA, USA: No Starch Press, 2019.
- [9] H. Lipovaca, *Learn You a Haskell for Great Good!: A Beginner’s Guide*. San Francisco, CA, USA: No Starch Press, 2011.
- [10] R. Leshchinskiy and contributors, *Haskell 2010 Language Report*. Haskell.org, 2010. [Online]. Available: <https://www.haskell.org/onlinereport/haskell2010/>

# Apêndice A

## Código-Fonte Completo

O código-fonte completo do Sistema de Estatística de Listas, conforme implementado e submetido ao repositório GitHub do Grupo 4, é apresentado a seguir.

```
1  -- =====
2  -- Ficheiro: script.hs
3  -- Projecto: Sistema de Estatística de Listas
4  -- Grupo: Grupo 4
5  -- Disciplina: [INSERIR DISCIPLINA]
6  -- Universidade: [NOME DA UNIVERSIDADE]
7  -- Ano: 2026
8  -- =====
9
10 module Main where
11
12 -- Calcula a soma de todos os elementos da lista
13 calcularSoma :: [Double] -> Double
14 calcularSoma [] = 0
15 calcularSoma xs = sum xs
16
17 -- Calcula a media aritmetica da lista
18 calcularMedia :: [Double] -> Double
19 calcularMedia xs = calcularSoma xs / fromIntegral (length xs)
20
21 -- Devolve o maior valor da lista
22 obterMaior :: [Double] -> Double
23 obterMaior xs = maximum xs
24
25 -- Devolve o menor valor da lista
26 obterMenor :: [Double] -> Double
27 obterMenor xs = minimum xs
28
29 -- Devolve a quantidade de elementos da lista
```

```
30 contarElementos :: [Double] -> Int
31 contarElementos xs = length xs
32
33 -- Funcao principal
34 main :: IO ()
35 main = do
36     putStrLn "===== "
37     putStrLn "Sistema de Estatistica de listas"
38     putStrLn "===== "
39     entrada <- getLine
40     let listaNumeros = map read (words entrada) :: [Double]
41     putStrLn "===== RESULTADOS ===== "
42     if null listaNumeros
43     then putStrLn "Erro: A lista nao pode estar vazia!"
44     else do
45         putStrLn "\n--- Resultados ---"
46         putStrLn ("0. Lista de numeros: "
47             ++ show listaNumeros)
48         putStrLn ("1. Soma: "
49             ++ show (calcularSoma listaNumeros))
50         putStrLn ("2. Media: "
51             ++ show (calcularMedia listaNumeros))
52         putStrLn ("3. Maior Valor: "
53             ++ show (obterMaior listaNumeros))
54         putStrLn ("4. Menor Valor: "
55             ++ show (obterMenor listaNumeros))
56         putStrLn ("5. Quantidade de elementos : "
57             ++ show (contarElementos listaNumeros))
```

Código A.1: Código-fonte completo – script.hs

## Apêndice B

# Lista Completa de Comandos Linux e Git Utilizados

Tabela B.1: Comandos Linux e Git utilizados no projecto

| Comando                                | Descrição                                                                                         |
|----------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>Comandos Linux</b>                  |                                                                                                   |
| <code>pwd</code>                       | Imprime o directório de trabalho actual ( <i>Print Working Directory</i> ).                       |
| <code>mkdir SisEstatistica</code>      | Cria o directório <code>SisEstatistica</code> no local actual.                                    |
| <code>cd SisEstatistica</code>         | Muda o directório de trabalho para <code>SisEstatistica</code> .                                  |
| <code>ls</code>                        | Lista os ficheiros e directórios do directório actual.                                            |
| <code>touch script.hs README.md</code> | Cria os ficheiros <code>script.hs</code> e <code>README.md</code> vazios.                         |
| <code>ghc -o programa script.hs</code> | Compila o ficheiro <code>script.hs</code> com o GHC, gerando o executável <code>programa</code> . |
| <code>./programa</code>                | Executa o programa compilado <code>programa</code> no directório actual.                          |
| <b>Comandos Git</b>                    |                                                                                                   |
| <code>git init</code>                  | Inicializa um repositório Git vazio no directório actual.                                         |
| <code>git add .</code>                 | Adiciona todas as alterações do directório actual à área de preparação ( <i>staging area</i> ).   |
| <code>git add script.hs</code>         | Adiciona especificamente o ficheiro <code>script.hs</code> à área de preparação.                  |

Continua na página seguinte...

---

| Comando                               | Descrição                                                                          |
|---------------------------------------|------------------------------------------------------------------------------------|
| <code>git add README.md</code>        | Adiciona o ficheiro <code>README.md</code> à área de preparação.                   |
| <code>git commit -m "mensagem"</code> | Regista um <i>commit</i> com a mensagem descritiva especificada.                   |
| <code>git checkout -b fase1</code>    | Cria a <i>branch</i> <code>fase1</code> e muda imediatamente para ela.             |
| <code>git checkout -b fase2</code>    | Cria a <i>branch</i> <code>fase2</code> e muda imediatamente para ela.             |
| <code>git checkout -b fim</code>      | Cria a <i>branch</i> <code>fim</code> e muda imediatamente para ela.               |
| <code>git checkout master</code>      | Muda para a <i>branch</i> <code>master</code> .                                    |
| <code>git merge fase1</code>          | Integra as alterações da <i>branch</i> <code>fase1</code> na <i>branch</i> actual. |
| <code>git merge fase2</code>          | Integra as alterações da <i>branch</i> <code>fase2</code> na <i>branch</i> actual. |
| <code>git merge fim</code>            | Integra as alterações da <i>branch</i> <code>fim</code> na <i>branch</i> actual.   |
| <code>git status</code>               | Exibe o estado actual da área de trabalho e da área de preparação.                 |
| <code>git log</code>                  | Apresenta o histórico de <i>commits</i> do repositório.                            |
| <code>git branch</code>               | Lista as <i>branches</i> existentes no repositório local.                          |

---